

Flui

Enterprise Challenge — Charge Map Cup — Etapa 2 — App funcional

Integrantes

Nome completo	RM
Bruna Pereira Cordeiro	563153
Lucas Buzato	562332

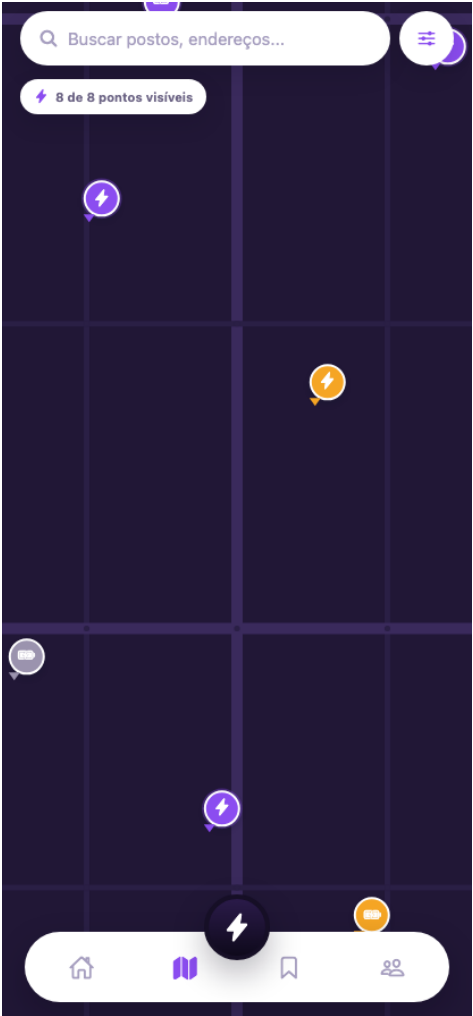
Integrantes listados em ordem alfabética pelo primeiro nome.

Capturas de Tela do App

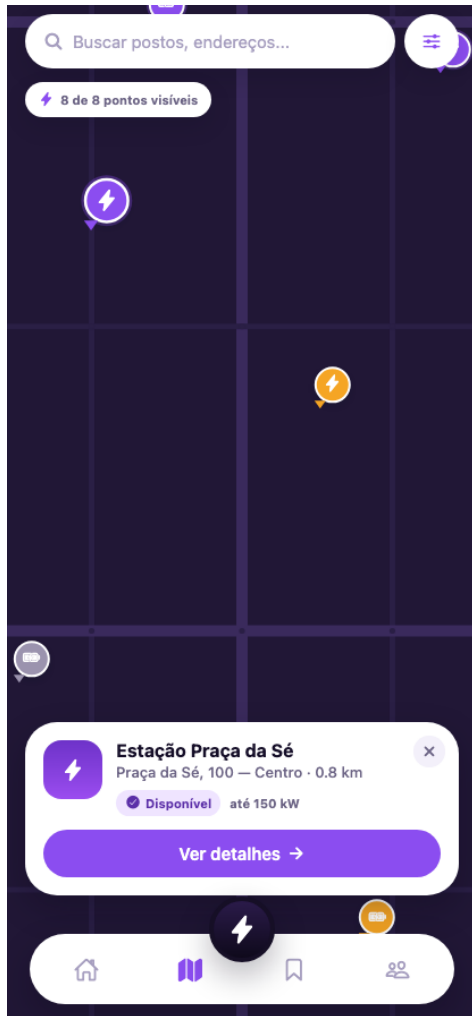
Prints reais do app rodando, capturados diretamente da build de desenvolvimento.



Início — dashboard com postos em destaque



Mapa interativo com marcadores diferenciados



Prévia rápida ao tocar em um marcador



Ficha detalhada do ponto de recarga



Filtros de busca funcionando (100+ kW aplicado)



Comunidade — feed de avaliações (diferencial)

←

Contar experiência

Sua opinião ajuda a comunidade

Selecione um posto

Escreva o endereço ou nome do posto

Como foi sua experiência geral?

🤔 😐 😊 😄 🥳

Avalie os principais aspectos

Disponibilidade
Fácil encontrar um carregador livre? ☆ ☆ ☆ ☆ ☆

Fila / Espera
Quanto tempo você esperou na fila? ☆ ☆ ☆ ☆ ☆

Velocidade de recarga
A recarga foi tão rápida quanto esperado? ☆ ☆ ☆ ☆ ☆

Conforto do local
Espaço limpo, seguro e agradável? ☆ ☆ ☆ ☆ ☆

Preço / Custo-benefício
O preço cobrado valeu a pena? ☆ ☆ ☆ ☆ ☆

Conte mais sobre sua experiência

Escreva aqui sua experiência, dicas ou observações...

0/500

Contar experiência (diferencial)

Justificativa das Tomadas de Decisão

Stack: React Native + Expo

Optamos por Expo em vez de React Native puro para acelerar o ciclo de prototipagem (hot reload, módulos nativos pré-configurados como mapa, gestos e animações) sem abrir mão de gerar um build nativo real ao final, via EAS Build.

Identidade visual: tema escuro em roxo/violeta

A paleta escura com gradientes roxo-violeta (fundo quase preto, cards brancos arredondados, acentos em roxo elétrico) foi escolhida para remeter a tecnologia e energia — conectando visualmente o tema de recarga elétrica à sensação de fluidez que dá nome ao app. A barra de navegação flutuante com botão central de raio (FAB) reforça essa identidade em todas as telas.

Mapa interativo com dados simulados + Google Maps real

Como o edital permite dados simulados, os pontos de recarga (localização, conectores, potência, ocupação) são mockados. Já a integração com o provedor de mapa é real: o app usa react-native-maps com o Google Maps SDK (Android/iOS), configurado via chave de API restrita por aplicativo. Para agilizar o desenvolvimento e testes no navegador durante a construção, foi criada

uma camada de compatibilidade que usa um mapa estilizado equivalente na web (onde bibliotecas de mapa nativo não rodam), mantendo os mesmos marcadores, cores e interações de toque/zoom/arraste.

Marcadores diferenciados por status e potência

Cada ponto no mapa usa cor (disponível, ocupado, indisponível) e ícone (raio para carga rápida $\geq 100\text{kW}$, bateria para os demais) simultaneamente — nunca só cor — já pensando em acessibilidade e leitura rápida.

Ficha do ponto rica e organizada

Além do mínimo pedido (carregadores, conectores, potência, horários, comodidades), foi adicionado um gráfico de ocupação por hora que calcula automaticamente o período de menor movimento, ajudando o motorista a planejar melhor a parada.

Filtros com estado compartilhado

Os filtros (tipo de conector, potência mínima, comodidades, horário de funcionamento) vivem num contexto global compartilhado entre a tela de Mapa e a tela de Filtros de Busca — aplicar um filtro em uma tela reflete imediatamente na outra, evitando que o usuário reconfigure a busca duas vezes.

Motion design aplicado de forma consistente

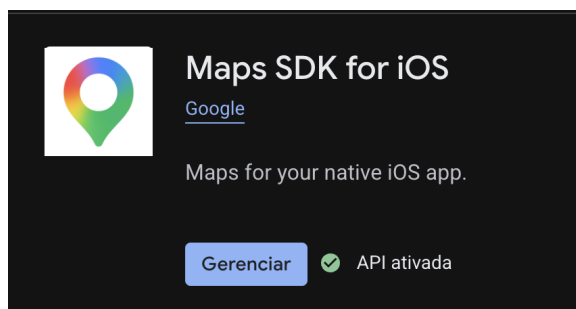
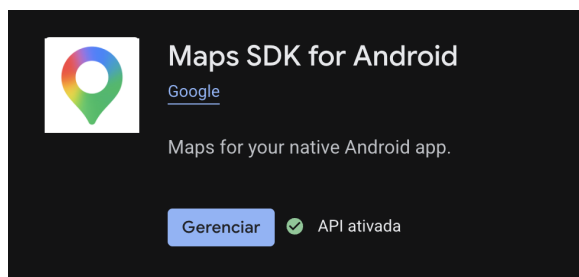
Splash screen animada na abertura, transições de tela (slide/fade) entre as telas da pilha, pulsação suave nos marcadores disponíveis do mapa, feedback animado de “Atualizando resultados...” ao aplicar filtros, entradas escalonadas (stagger) dos cartões de lista e um estado de carregamento animado ao abrir o mapa.

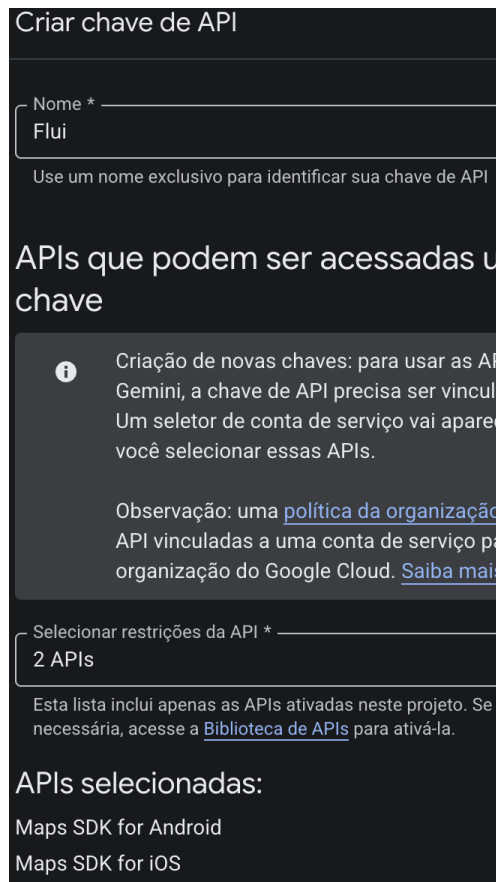
Funcionalidades extras (Favoritos e Comunidade)

Embora o edital indique que sistema de avaliação e favoritos não são exigidos nesta etapa, implementamos versões funcionais de ambos (favoritar um posto e publicar uma experiência com notas por categoria) para enriquecer a demonstração e adiantar trabalho da próxima etapa, sem que isso substitua os requisitos obrigatórios.

Integração com o Google Maps

O mapa interativo do Flui está integrado de verdade ao Google Maps — não é uma URL estática nem uma imagem, é o SDK oficial do Google embutido no app via react-native-maps. No Google Cloud Console do projeto, as APIs **Maps SDK for Android** e **Maps SDK for iOS** estão ativas, e a chave de API (nomeada “Flui”) foi criada restrita exatamente a essas duas APIs — conforme as capturas abaixo, feitas direto no painel do Google.





Criar chave de API

Nome *
Flui

Use um nome exclusivo para identificar sua chave de API

APIs que podem ser acessadas u
chave

i Criação de novas chaves: para usar as AP
Gemini, a chave de API precisa ser vincula
Um seletor de conta de serviço vai aparec
você selecionar essas APIs.

Observação: uma [política da organização](#)
API vinculadas a uma conta de serviço pa
organização do Google Cloud. [Saiba mais](#)

Selecionar restrições da API *
2 APIs

Esta lista inclui apenas as APIs ativadas neste projeto. Se v
necessária, acesse a [Biblioteca de APIs](#) para ativá-la.

APIs selecionadas:

Maps SDK for Android
Maps SDK for iOS

Chave de API “Flui” restrita às duas APIs de mapa

Como ver o mapa real funcionando

O Maps SDK é um módulo **nativo** — ele não roda em navegador nem no Expo Go, só dentro de um build instalado no celular. Por isso, para ver os tiles reais do Google (não a versão estilizada usada durante o desenvolvimento web), os passos são:

- 1 Gerar um build nativo do app com o EAS (`eas build --platform android` ou `--platform ios`), que já usa a chave de API acima.
- 2 Baixar e instalar o instalador gerado (.apk no Android, ou via TestFlight/simulador no iOS) no aparelho.
- 3 Abrir o app e ir até a aba Mapa — os pontos de recarga aparecem como marcadores sobre o mapa real do Google, com pan e zoom nativos.

Um build gerado dessa forma já está publicado e pode ser instalado diretamente por este link: <https://expo.dev/artifacts/eas/xHwWGi1V6sZcZ538t8u8fq1i0zuwXz2t6fwZCCmwIBI.apk>

Escopo desta Etapa

Conforme o edital, os itens abaixo **não** são exigência obrigatória da Etapa 2 e foram implementados apenas como diferencial: sistema de avaliação (tela Comunidade / Contar Experiência) e favoritos/histórico. Vídeo-pitch também não é exigido nesta entrega.

Repositório no GitHub

<https://github.com/rhunaa/ChallengeFlui>